IMPERIAL

Department of Mathematics
Imperial College London

Formalizing Group Action in Lean

Student name: *Frankie Feng-Cheng WANG*
Email: *maths@frankie.wang*
GitHub: *https://github.com/FrankieeW/GroupAction*

Course: *MATH70040-Formalising Mathematics* – Lecturer: *Dr Bhavik Mehta*
Due date: *January 29, 2026*

Contents

1. Introduction

This report presents a Lean 4 formalisation of **group actions**. The development is organised around three closely related components:

1. **Core definitions:** the `GroupAction` typeclass, together with the notions of faithfulness and transitivity;
2. **Concrete examples:** a selection of standard group actions illustrating the abstract definitions;
3. **Key theorems:** the construction of the permutation representation associated to a group action and the subgroup structure of stabilisers.

The mathematical development and theorem numbering follow Fraleigh and Katz, *A First Course in Abstract Algebra*, Addison–Wesley, 2003, Section 16 (Group Actions).
The complete Lean source code is available on GitHub:

`https://github.com/FrankieeW/GroupAction`

1.1. Running the Code. To verify the formalization, use Lake to build and run the Lean code:

```
lake build                        # Build the project
lake env lean lean/GroupAction.lean    # Run the main file
lake env lean lean/GroupAction/Examples.lean  # Check all examples
```

All code snippets appearing in this report are hyperlinked to the corresponding lines in the repository.
The principal results formalised in this project are stated below and proved in later sections.

Theorem 16.3. Let X be a G-set. For each $g \in G$, the map

$$\sigma_g : X \to X, \qquad \sigma_g(x) = g \cdot x,$$

is a permutation of X. Moreover, the assignment

$$\phi : G \to \mathrm{Sym}(X), \qquad \phi(g) = \sigma_g,$$

is a group homomorphism, and for all $g \in G$ and $x \in X$,

$$\phi(g)(x) = g \cdot x.$$

Theorem 16.12. Let X be a G-set and let $x \in X$. The stabiliser of x in G, defined by

$$G_x = \{\, g \in G \mid g \cdot x = x \,\},$$

is a subgroup of G.

2. Definitions

This section introduces the group action class and the definitions of faithful and transitive actions.

2.1. Group Action.

Now, we present the definition of a group action both mathematically and in Lean.

Mathematical definition.
A **group action** of a group G on a set X is a map

$$\cdot : G \times X \to X,$$

satisfying, for all $g_1, g_2 \in G$ and $x \in X$:

- (Compatibility) $(g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x)$;
- (Identity) $1 \cdot x = x$.

Lean formalisation.
The following Lean code defines a (left) action of a monoid G on a type X.

- `act : G → X → X` represents the action $g \cdot x$;
- `ga_mul` encodes compatibility with multiplication;
- `ga_one` states that the identity acts trivially.

```
20
21
22  /-! ## Definitions: act faithfully -/
23  /-- A group action is faithful if different group elements
```

Using `Monoid` instead of `Group` allows the action to be defined for monoids in general. The inverse element is not needed for the axioms. However, I will only consider group actions in this project.

```
13  variable {G : Type*} [Group G] {X : Type*} [GroupAction G X]
```

2.2. Faithful and transitive actions.

Faithful actions. An action of a group G on a set X is said to be **faithful** if distinct group elements induce distinct permutations of X. Equivalently, the action is faithful if

$$(\forall x \in X,\ g_1 \cdot x = g_2 \cdot x) \implies g_1 = g_2.$$

In other words, no nontrivial group element acts trivially on all of X.

```
  ∀ g₁ g₂ : G, (∀ x : X, GroupAction.act g₁ x = GroupAction.act g₂ x) → g₁
  ↪  = g₂

/-! ## Definitions: transitive -/
```

Transitive actions. An action of a group G on a set X is **transitive** if for any $x_1, x_2 \in X$, there exists an element $g \in G$ such that

$$g \cdot x_1 = x_2.$$

This means that X consists of a single orbit under the action of G.

```
  ∀ x₁ x₂ : X, ∃ g : G, GroupAction.act g x₁ = x₂
```

3. Concrete Examples of Group Actions

This section presents several concrete instances of the `GroupAction` class, illustrating how the abstract definition applies to familiar mathematical structures.

3.1. Symmetric group acting on a set. Let X be any type. The symmetric group $\mathrm{Sym}(X)$ acts on X by evaluation: for $\sigma \in \mathrm{Sym}(X)$ and $x \in X$, define $\sigma \cdot x := \sigma(x)$.

```
instance permGroupAction (X : Type*) : GroupAction (Equiv.Perm X) X :=
  { act := fun g x => g x
    ga_mul := by
      intro g1 g2 x
      rfl
    ga_one := by
      intro x
      rfl }
```

The axiom proofs are immediate by reflexivity: composition of permutations and function evaluation commute definitionally in Lean.
Two standard properties of this action are faithful and transitive:

```
theorem permGroupAction_faithful (X : Type*) :
  GroupAction.faithful (G := Equiv.Perm X) (X := X) :=
by
  intro g₁ g₂ h
  ext x
  exact h x

theorem permGroupAction_transitive
  (X : Type*) [DecidableEq X] :
  GroupAction.transitive (G := Equiv.Perm X) (X := X) :=
by
  intro x₁ x₂
  refine ⟨Equiv.swap x₁ x₂, ?_⟩
  simp [GroupAction.act]
--rmk: `[DecidableEq X]` is needed for `Equiv.swap`
```

The instance `DecidableEq X` is required because `Equiv.swap` constructs a permutation by branching on whether two elements are equal, which needs decidable equality.

3.2. Group acting on itself by left multiplication. Every group G acts on itself by left multiplication: $g_1 \cdot g_2 := g_1 * g_2$.

```
44 instance groupAsGSet (G : Type*) [Group G] : GroupAction G G :=
45   { act := fun g1 g2 => g1 * g2
46     ga_mul := by
47       intro g1 g2 g3
48       rw [mul_assoc]
49     ga_one := by
50       intro g
51       rw [one_mul] }
```

The `ga_mul` axiom follows from associativity of group multiplication, and `ga_one` from the identity axiom.

3.3. Subgroup acting on the group. A subgroup $H \leq G$ acts on G by left multiplication. Elements of H are coerced to elements of G using `(h : G)`.

```
55 instance subgroupAsGSet {G : Type*} [Group G] (H : Subgroup G) :
   ↪ GroupAction H G :=
56   { act := fun h g => (h : G) * g
57     ga_mul := by
58       intros
59       -- simp
60       -- rw [mul_assoc]
61       simp [mul_assoc]
62     ga_one := by
63       intros
64       simp }
```

The `simp` tactic handles the coercion and applies associativity and identity axioms.

3.4. Conjugation action of a subgroup on itself. A subgroup H acts on itself by conjugation: $h_1 \cdot h_2 := h_1 h_2 h_1^{-1}$.

```
66 instance subgroupAsGSetConjugation {G : Type*} [Group G] (H : Subgroup G)
   ↪ : GroupAction H H :=
67   {
68     act := fun h g => h * g * h⁻¹
69
70     ga_mul := by
71       intro g₁ g₂ g₃
72       -- one step
73       -- simp [mul_assoc]
74       -- in detail
75       simp
76       -- goal state: g₁ * g₂ * g₃ * (g₂⁻¹ * g₁⁻¹) = g₁ * (g₂ * g₃ * g₂⁻¹) *
          ↪ g₁⁻¹
77       rw [← mul_assoc g₁]
```

```
78        -- goal state: g₁ * g₂ * g₃ * (g₂⁻¹ * g₁⁻¹) = g₁ * (g₂ * g₃) * g₂⁻¹ *
          ↪ g₁⁻¹
79        rw [mul_assoc g₁ g₂]
80        -- goal state: g₁ * (g₂ * g₃) * (g₂⁻¹ * g₁⁻¹) = g₁ * (g₂ * g₃) *
          ↪ g₂⁻¹ * g₁⁻¹
81        rw [← mul_assoc]
82      ga_one := by
83        intros
84        simp }
```

The `mul_assoc` lemma is from mathlib and states associativity of multiplication:

```
1  mul_assoc.{u_1} {G : Type u_1} [Semigroup G] (a b c : G) :
2  a * b * c = a * (b * c)
```

The `ga_mul` proof can easily be done by `simp [mul_assoc]`, but I expanded it to show the steps explicitly by using `rw`.
For example the `rw [← mul_assoc g₁]` will aotumatically find the passible situation looks like `g₁* (b * c)`, because we only give one argument `g₁` to `← mul_assoc`. In detail, it will rewrite the left hand side `g₁ * (g₂ * g₃ * g₂⁻¹)` to `g₁ * (g₂ * g₃) * g₂⁻¹` according to the right to left direction of the lemma `mul_assoc`.

3.5. Scalar action on complex vector spaces. The multiplicative group $\mathbb{C}^\times$ of nonzero complex numbers acts on $\mathbb{C}^n$ by scalar multiplication.

```
88  instance vectorSpaceAsCStarSet (n : ℕ) :
89      GroupAction (ℂˣ) (Fin n → ℂ) :=
90    { act := fun r v => fun i => (r : ℂ) * v i
91      ga_mul := by
92        intros r1 r2 v
93        ext i
94        simp [mul_assoc]
95      ga_one := by
96        intro v
97        ext i
98        simp }
```

Here `ℂˣ` denotes the units of $\mathbb{C}$ (invertible elements), and `Fin n → ℂ` represents functions from $\{0, \ldots, n-1\}$ to $\mathbb{C}$ (i.e., n-dimensional complex vectors). The `ext` tactic proves function equality pointwise.

3.6. Scalar action via units. The unit group $\alpha^\times$ of a monoid α acts on α^n by scalar multiplication.

```
103 instance vectorSpaceAsUnitSet
104   (α : Type*) [Monoid α]
105   (n : ℕ) :
106   GroupAction (αˣ) (Fin n → α) :=
107 { act := fun r v => fun i => (r : α) * v i
108   ga_mul := by
109     intros r1 r2 v
```

```
    ext i
    simp [mul_assoc]
  ga_one := by
    intro v
    ext i
    simp }
```

This generalizes the complex example by abstracting over the coefficient monoid.

3.7. Dihedral group action on a square. Let D_4 be the symmetry group of a square, acting on $\mathbb{Z}/4\mathbb{Z}$.

```
abbrev D4 := DihedralGroup 4
abbrev Vertex := ZMod 4
abbrev Side := ZMod 4
abbrev Midpoint := ZMod 4

def d4Act (g : D4) (x : ZMod 4) : ZMod 4 :=
  match g with
  -- Rotation r_i: x ↦ x - i
  | DihedralGroup.r i => x - i
  -- Reflection s_i: x ↦ i - x
  | DihedralGroup.sr i => i - x

instance d4ActionZMod4 : GroupAction D4 (ZMod 4) :=
  { act := d4Act
    ga_mul := by
      intro g1 g2 x
      cases g1 <;> cases g2 <;>
        simp [d4Act, sub_eq_add_neg] <;> ring
    ga_one := by
      intro x
      simp [DihedralGroup.one_def, d4Act, -DihedralGroup.r_zero]
    }
```

3.8. Symmetric group S acting on {1,2,3}. The symmetric group on 3 elements provides the canonical example of a transitive group action. This is the standard permutation representation of S_3.

```
/-! ## Example: Symmetric group S₃ acting on {1,2,3} -/
/-!
The symmetric group on 3 elements acts transitively on `Fin 3`.
This is the standard example of a transitive group action.
-/
abbrev S3 := Equiv.Perm (Fin 3)

instance s3ActionFin3 : GroupAction S3 (Fin 3) :=
  { act := fun g x => g x
    ga_mul := by
      intro g1 g2 x
      rfl
    ga_one := by
      intro x
```

```
180       rfl }
181
```

The action is transitive: for any two elements $a, b \in \{1, 2, 3\}$, there exists a permutation $\sigma \in S_3$ such that $\sigma(a) = b$.

4. Permutation Representation

The core construction is the map $\sigma_g : X \to X$ given by $\sigma_g(x) = g \cdot x$. The inverse of σ_g is $\sigma_{g^{-1}}$, so σ_g is a permutation. This yields the permutation representation $\phi : G \to \mathrm{Sym}(X)$.

4.1. Proof Path. To establish the permutation representation, we follow these steps:

1. **Define the action map:** For each $g \in G$, construct $\sigma_g : X \to X$ by $\sigma_g(x) = g \cdot x$.
2. **Prove σ_g is a bijection:** Show that σ_g has an inverse, namely $\sigma_{g^{-1}}$.
3. **Construct the permutation:** Package σ_g as an element of $\mathrm{Sym}(X)$ using the inverse.
4. **Define the representation:** Set $\phi(g) = \sigma_g$ to get $\phi : G \to \mathrm{Sym}(X)$.
5. **Verify homomorphism properties:** Prove that ϕ preserves multiplication and the identity.

4.2. Step-by-Step Proof with Mathematical and Code Explanations. We now walk through each step in detail, providing both mathematical reasoning and the corresponding Lean code.

4.2.1. Step 1: Define the action map σ_g. This step may not be strictly necessary in Lean code, but we include it to align with traditional mathematical proof structure, showing how the action map is constructed before proving its properties.

Mathematical explanation.
For each group element $g \in G$, we define a function $\sigma_g : X \to X$ that applies the group action: $\sigma_g(x) = g \cdot x$. This is simply the action function partially applied to g.

Lean code explanation.
The function `sigma` implements this definition. It takes a group element `g` and returns a function from `X` to `X` that applies the action.

```
26
27  /-- Defines the action map `σ_g : X → X` by `x ↦ g • x`. -/
28  def sigma (g : G) : X → X :=
29    fun x => GroupAction.act g x
```

4.2.2. Step 2: Prove σ_g is a bijection. Lean requires an explicit construction of the inverse function to show that σ_g is a permutation.

Mathematical explanation.
To show σ_g is bijective, we need both injectivity and surjectivity. The key insight is that the inverse operation g^{-1} provides the inverse function: $\sigma_{g^{-1}} \circ \sigma_g = \mathrm{id}_X$ and $\sigma_g \circ \sigma_{g^{-1}} = \mathrm{id}_X$. This follows from the group axioms and the action compatibility.

Lean code explanation.
The `sigmaPerm` definition constructs a permutation by providing both the forward function (`toFun := sigma g`) and its inverse (`invFun := sigma g⁻¹`). The proofs of `left_inv` and `right_inv` verify the two-sided inverse properties using the action axioms.

```
31
32  /-- Defines the permutation of `X` induced by `g`.
33      The inverse is given by the action of `g⁻¹`. -/
34  def sigmaPerm (g : G) : Equiv.Perm X :=
35    { toFun := sigma g
36      invFun := sigma g⁻¹
37      left_inv := by
38        intro x
39        -- Step 1: combine `g⁻¹` and `g` using the action axiom.
40        calc
41          GroupAction.act g⁻¹ (GroupAction.act g x) =
42              GroupAction.act (g⁻¹ * g) x := by
43            simpa using (GroupAction.ga_mul g⁻¹ g x).symm
44          -- Step 2: simplify `g⁻¹ * g` to `1`.
45          _ = GroupAction.act (1 : G) x := by
46            -- Alternative: use `congrArg` with `inv_mul_cancel g`.
47            -- congrArg (fun t => GroupAction.act t x) (inv_mul_cancel g)
48            simp
49          -- Step 3: the identity acts as the identity on `X`.
50          _ = x := GroupAction.ga_one x
51      right_inv := by
52        intro x
53        -- Step 1: combine `g` and `g⁻¹` using the action axiom.
54        calc
55          GroupAction.act g (GroupAction.act g⁻¹ x) =
56              GroupAction.act (g * g⁻¹) x := by
57            simpa using (GroupAction.ga_mul g g⁻¹ x).symm
58          -- Step 2: simplify `g * g⁻¹` to `1`.
59          _ = GroupAction.act (1 : G) x := by
60            -- Alternative: use `congrArg` with `mul_inv_cancel g`.
61            -- congrArg (fun t => GroupAction.act t x) (mul_inv_cancel g)
62            simp
63          -- Step 3: the identity acts as the identity on `X`.
```

4.2.3. Step 3: Construct the permutation representation ϕ. This step bridges the individual permutations σ_g to a group homomorphism, constructing the representation map $\phi : G \to \mathrm{Sym}(X)$ that preserves the group structure.

Mathematical explanation.
Having established that each σ_g is a permutation, we can define the representation map $\phi : G \to \mathrm{Sym}(X)$ by $\phi(g) = \sigma_g$. This assignment preserves the action: $\phi(g)(x) = \sigma_g(x) = g \cdot x$.

Lean code explanation.
The function `phi` simply wraps `sigmaPerm`, and `phi_apply` confirms that it agrees with the original action.

```
66
67  /-- Defines the permutation representation `phi : G → Equiv.Perm X`.
68      This sends `g` to the permutation `σ_g`. -/
69  def phi (g : G) : Equiv.Perm X :=
70    sigmaPerm g
71
72  /-- `phi` agrees with the action on each element of `X`. -/
```

4.2.4. Step 4: Verify homomorphism properties. This step establishes that ϕ is a group homomorphism by verifying it preserves multiplication and the identity element, completing the proof that ϕ is a valid group representation.

Mathematical explanation.
For ϕ to be a group homomorphism, we need $\phi(g_1g_2) = \phi(g_1)\phi(g_2)$ and $\phi(1) = 1$. The multiplication property follows from the action compatibility axiom: $\phi(g_1g_2)(x) = (g_1g_2) \cdot x = g_1 \cdot (g_2 \cdot x) = \phi(g_1)(\phi(g_2)(x))$.

Lean code explanation.
The lemmas `phi_mul` and `phi_one` prove these properties. `phi_mul` uses `Equiv.ext` for extensionality and applies the `ga_mul` axiom, while `phi_one` uses the `ga_one` axiom.

```
78
79  /-- Shows that `phi` respects multiplication:
80      `phi (g₁ * g₂) = phi g₁ * phi g₂`. -/
81  lemma phi_mul (g₁ g₂ : G) :
82    phi (X := X) (g₁ * g₂) = phi (X := X) g₁ * phi (X := X) g₂ := by
83    -- Extensionality reduces equality of permutations to pointwise equality.
84    apply Equiv.ext
85    intro (x : X)
86    calc
87      -- Step 1: expand `phi` and the action definition.
88      phi (g₁ * g₂) x = GroupAction.act (g₁ * g₂) x := rfl
89      -- Step 2: use the action axiom to separate `g₁` and `g₂`.
90      _ = GroupAction.act g₁ (GroupAction.act g₂ x) := GroupAction.ga_mul g₁
        ↪  g₂ x
91
92  /-- `phi 1` is the identity permutation. -/
93  lemma phi_one : phi (1 : G) = (1 : Equiv.Perm X) := by
94    apply Equiv.ext
95    intro (x : X)
96    calc
97      phi (1 : G) x = GroupAction.act (1 : G) x := rfl
98      _ = x := GroupAction.ga_one x
99      _ = (1 : Equiv.Perm X) x := by
```

4.2.5. Step 5: Package the theorem. Now just need to combine everything into the final theorem statement.

Mathematical explanation.
We now have all the ingredients: a homomorphism ϕ that satisfies the required properties. This completes the construction of the permutation representation.

Lean code explanation.
The theorem `group_action_to_perm_representation` packages everything together, using the previously proved lemmas to construct the existential witness.

```
101
102 /-- The action yields a permutation representation. -/
103 theorem group_action_to_perm_representation :
104   ∃ (ψ : G → Equiv.Perm X),
105     (∀ g x, ψ g x = GroupAction.act g x) ∧
106     (∀ g₁ g₂, ψ (g₁ * g₂) = ψ g₁ * ψ g₂) ∧
107     (ψ 1 = 1) := by
```

4.3. Proof sketch (informal). To show that σ_g is a bijection, compute $\sigma_{g^{-1}}(\sigma_g(x)) = (g^{-1}g) \cdot x = 1 \cdot x = x$, and similarly $\sigma_g(\sigma_{g^{-1}}(x)) = x$. The representation is defined by $\phi(g) = \sigma_g$, and multiplicativity follows from the action axiom:

$$\phi(g_1 g_2)(x) = (g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x) = (\phi(g_1)\phi(g_2))(x).$$

4.4. Lean code. First, define the underlying action map:

```
26
27 /-- Defines the action map `σ_g : X → X` by `x ↦ g • x`. -/
28 def sigma (g : G) : X → X :=
29   fun x => GroupAction.act g x
```

Then package it as a permutation using the inverse action:

```
31
32 /-- Defines the permutation of `X` induced by `g`.
33     The inverse is given by the action of `g⁻¹`. -/
34 def sigmaPerm (g : G) : Equiv.Perm X :=
35   { toFun := sigma g
36     invFun := sigma g⁻¹
37     left_inv := by
38       intro x
39       -- Step 1: combine `g⁻¹` and `g` using the action axiom.
40       calc
41         GroupAction.act g⁻¹ (GroupAction.act g x) =
42             GroupAction.act (g⁻¹ * g) x := by
43           simpa using (GroupAction.ga_mul g⁻¹ g x).symm
44         -- Step 2: simplify `g⁻¹ * g` to `1`.
45         _ = GroupAction.act (1 : G) x := by
46           -- Alternative: use `congrArg` with `inv_mul_cancel g`.
47           -- congrArg (fun t => GroupAction.act t x) (inv_mul_cancel g)
48           simp
49         -- Step 3: the identity acts as the identity on `X`.
```

```
        _ = x := GroupAction.ga_one x
    right_inv := by
      intro x
      -- Step 1: combine `g` and `g⁻¹` using the action axiom.
      calc
        GroupAction.act g (GroupAction.act g⁻¹ x) =
            GroupAction.act (g * g⁻¹) x := by
          simpa using (GroupAction.ga_mul g g⁻¹ x).symm
        -- Step 2: simplify `g * g⁻¹` to `1`.
        _ = GroupAction.act (1 : G) x := by
          -- Alternative: use `congrArg` with `mul_inv_cancel g`.
          -- congrArg (fun t => GroupAction.act t x) (mul_inv_cancel g)
          simp
        -- Step 3: the identity acts as the identity on `X`.
```

Using `#check` on `sigmaPerm` confirms it has type

```
        _ = x := GroupAction.ga_one x }
```

This confirms the map $g \mapsto \sigma_g$ is a well-defined function from G to $\mathrm{Sym}(X)$. Define the representation and its basic properties:

```

/-- Defines the permutation representation `phi : G → Equiv.Perm X`.
    This sends `g` to the permutation `σ_g`. -/
def phi (g : G) : Equiv.Perm X :=
  sigmaPerm g

/-- `phi` agrees with the action on each element of `X`. -/
lemma phi_apply (g : G) (x : X) : phi g x = GroupAction.act g x := rfl

-- /-- A commented-out attempt at proving multiplicativity of `phi`. -/
-- lemma phi_mul : ∀ (g₁ g₂ : G), phi (g₁ * g₂) = phi g₁ * phi g₂ := by
--   sorry

/-- Shows that `phi` respects multiplication:
    `phi (g₁ * g₂) = phi g₁ * phi g₂`. -/
lemma phi_mul (g₁ g₂ : G) :
  phi (X := X) (g₁ * g₂) = phi (X := X) g₁ * phi (X := X) g₂ := by
  -- Extensionality reduces equality of permutations to pointwise equality.
  apply Equiv.ext
  intro (x : X)
  calc
    -- Step 1: expand `phi` and the action definition.
    phi (g₁ * g₂) x = GroupAction.act (g₁ * g₂) x := rfl
    -- Step 2: use the action axiom to separate `g₁` and `g₂`.
    _ = GroupAction.act g₁ (GroupAction.act g₂ x) := GroupAction.ga_mul g₁
     ↪  g₂ x

/-- `phi 1` is the identity permutation. -/
lemma phi_one : phi (1 : G) = (1 : Equiv.Perm X) := by
  apply Equiv.ext
  intro (x : X)
  calc
    phi (1 : G) x = GroupAction.act (1 : G) x := rfl
    _ = x := GroupAction.ga_one x
```

```
    _ = (1 : Equiv.Perm X) x := by
```

Finally, package the existence statement:

```

/-- The action yields a permutation representation. -/
theorem group_action_to_perm_representation :
  ∃ (ψ : G → Equiv.Perm X),
    (∀ g x, ψ g x = GroupAction.act g x) ∧
    (∀ g₁ g₂, ψ (g₁ * g₂) = ψ g₁ * ψ g₂) ∧
    (ψ 1 = 1) := by
```

5. Stabilizers

For a fixed $x \in X$, the stabilizer is the subset

$$G_x = \{g \in G \mid g \cdot x = x\}.$$

In Lean this is first defined as a set, then shown to be the carrier of a subgroup, and finally packaged as a `Subgroup`.

5.1. Proof sketch (informal). The identity element fixes every x, so $1 \in G_x$. If g_1 and g_2 fix x, then $(g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x) = g_1 \cdot x = x$, so $g_1 g_2 \in G_x$. If g fixes x, then $g^{-1} \cdot x = g^{-1} \cdot (g \cdot x) = (g^{-1} g) \cdot x = x$, so $g^{-1} \in G_x$.

5.2. Lean code. Start from the set definition:

```
/-- The stabilizer set
    `G_x = { g ∈ G | g • x = x }`. -/
def stabilizerSet (x : X) : Set G :=
  { g : G | GroupAction.act g x = x }
```

Package it as a subgroup by checking closure: Define the subgroup structure by providing proofs of the subgroup axioms:

- `carrier`: the underlying set is `stabilizerSet x`.
- `one_mem'`: the identity fixes every x by the `ga_one` axiom.
- `mul_mem'`: if g_1 and g_2 fix x, then so does $g_1 g_2$ by the `ga_mul` axiom.
- `inv_mem'`: if g fixes x, then so does g^{-1} by combining `ga_mul` and `ga_one`.

```
/-- The stabilizer `G_x` as a subgroup of `G`. -/
def stabilizer (x : X) : Subgroup G := by
  exact
    { carrier := stabilizerSet (G := G) (X := X) x
      one_mem' := by
        -- The identity fixes every x.
        simp [stabilizerSet, GroupAction.ga_one x]
      mul_mem' := by
        intro g₁ g₂ hg₁ hg₂
```

```
        calc
          -- Closure under multiplication via the action axiom.
          GroupAction.act (g₁ * g₂) x = GroupAction.act g₁
          ↪  (GroupAction.act g₂ x) := by
            simpa using (GroupAction.ga_mul g₁ g₂ x)
          _ = GroupAction.act g₁ x := by
            rw [hg₂]
          _ = x := hg₁
      inv_mem' := by
        intro g hg
        calc
          -- If g fixes x, then g⁻¹ fixes x as well.
          GroupAction.act g⁻¹ x = GroupAction.act g⁻¹ (GroupAction.act g
          ↪  x) := by
            rw [hg]
          _ = GroupAction.act (g⁻¹ * g) x := by
            simpa using (GroupAction.ga_mul g⁻¹ g x).symm
          _ = GroupAction.act (1 : G) x := by simp
          _ = x := GroupAction.ga_one x }
```

Then show the set is the carrier of a subgroup:

```
/-- The stabilizer set is the carrier of a subgroup of `G`. -/
theorem stabilizer_set_is_subgroup (x : X) :
    ∃ H : Subgroup G, (H : Set G) = stabilizerSet (G := G) (X := X) x := by
  refine ⟨stabilizer (G := G) (X := X) x, rfl⟩
```

6. What Lean Guarantees

Formalizing these constructions in Lean provides guarantees that informal proofs cannot match. When Lean accepts a definition or theorem, it has verified:

- **Types are correct.** The function `phi : G → Equiv.Perm X` has the claimed type. Lean checks that `sigmaPerm g` is actually a permutation (a bijection), not just a function. If we mistakenly defined a non-bijective map, Lean would reject it.
- **No hidden assumptions.** Every lemma explicitly lists its hypotheses. If a proof uses associativity, the code must invoke `mul_assoc` or the `ga_mul` axiom. There are no "clearly" or "it follows that" steps that skip justification.
- **Axioms match definitions.** The `GroupAction` class requires `ga_mul` and `ga_one`. If we omitted `ga_one`, the proof of `phi_one` would fail because Lean would have no way to show $1 \cdot x = x$.
- **Subgroup structure is verified.** When we package the stabilizer as a `Subgroup`, Lean checks that we provided proofs of closure under multiplication, inverses, and identity. The type system prevents us from claiming "the stabilizer is a subgroup" without proving it.

These checks prevent common errors: mistaken claims about injectivity, forgotten hypotheses, and gaps in subgroup proofs. An external examiner can trust that the Lean code genuinely establishes the mathematical claims, not just that it compiles.

7. Reflection

Writing the proofs in Lean forced me to make every step explicit. In particular:

- I had to show explicitly that $\sigma_{g^{-1}}$ is an inverse.
- I had to unfold the definition of permutation multiplication.

The resulting code is not shorter than a textbook proof, but it is more precise. An external examiner can read the surrounding explanations and then see that each Lean line matches a specific mathematical claim.

7.1. Specific challenges encountered. Several technical hurdles emerged during the formalization:

- **Typeclass resolution:** Ensuring Lean could automatically find the `GroupAction` instance in complex contexts required careful use of explicit type annotations (e.g., `(G := G)`).
- **Equivalence mechanics:** Constructing `sigmaPerm` as an `Equiv.Perm X` required providing both `left_inv` and `right_inv` proofs, which meant carefully applying the `ga_mul` axiom in both directions.
- **Coercions:** Working with subgroups required understanding how Lean coerces elements of type `H` (a subgroup) to type `G` (the ambient group) using `(h : G)`.
- **Function extensionality:** Proving equality of permutations required `Equiv.ext`, and equality of vector-valued functions required `ext i`, which were not immediately obvious.

8. Relation to Mathlib's `MulAction`

Mathlib's `MulAction` class provides a general framework for monoid actions, which is mathematically equivalent to our `GroupAction` class. The key differences are philosophical and structural rather than mathematical:

- **Mathematical equivalence**: Both classes define actions $G \times X \to X$ satisfying $(g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x)$ and $1 \cdot x = x$.
- **Type class requirements**: `MulAction` requires a `Monoid` G, while our `GroupAction` also requires a `Monoid` G (technically, this makes them equivalent).
- **Emphasis on groups**: Our `GroupAction` class is designed specifically for group actions, with definitions like faithfulness and transitivity that assume group properties (e.g., inverses).
- **Extensibility**: The custom class allows us to add group-specific theorems and examples without modifying Mathlib's general-purpose classes.

This custom class serves as a foundation for group theory development while remaining compatible with Mathlib's ecosystem.

9. Future Extensions

This formalization provides a foundation for further development in group action theory. Potential extensions include: the orbit-stabilizer theorem relating the size of orbits to cosets of stabilizers; Burnside's lemma for counting group elements with fixed points; group actions on cosets, leading to the concept of permutation representations; and applications to representation theory and algebraic topology.

10. AI Assistance Statement

I used AI-assisted tools in a limited and strictly auxiliary manner. Specifically, AI was used to help automate the conversion of Lean source files into LaTeX code blocks for inclusion in this report, and to assist with reorganising a single Lean file into multiple smaller files during the development process. In addition, AI was occasionally used for high-level brainstorming about structure and presentation.
All mathematical content, Lean definitions, proofs, and formal statements were written, checked, and verified by me. AI tools were not used to replace mathematical reasoning or to generate unverified results. I take full responsibility for the correctness and originality of both the Lean code and the written exposition.

11. Lean Tactics Glossary

This section provides a brief explanation of the key Lean tactics used in the proofs throughout this formalization.

- **`intro`**: Introduces assumptions into the local context. For example, `intro g x` brings a group element g and a set element x into scope for a proof.
- **`apply`**: Applies a lemma or theorem to the current goal. For example, `apply ga_mul` uses the group action multiplication axiom.
- **`simp`**: Simplifies expressions using equational lemmas and basic rules. For example, `simp` might simplify $(g_1 g_2) \cdot x$ to $g_1 \cdot (g_2 \cdot x)$ using associativity.
- **`rw`**: Rewrites the goal or hypotheses using an equality. For example, `rw [ga_one]` rewrites using the identity axiom.
- **`calc`**: Structures multi-step equalities in a readable way. For example, `calc` blocks break down complex equality proofs into clear steps.
- **`ext`**: Proves equality of functions by proving they agree on all inputs. For example, `ext x` proves two functions are equal by showing they produce the same output for every x.
- **`exact`**: Closes a goal by providing an exact term that matches the goal type. For example, `exact rfl` proves $x = x$.
- **`refine`**: Partially constructs a term, leaving holes for subgoals. For example, `refine { act := fun g x => g x, ga_mul := ?_ }` constructs a GroupAction instance with a hole for the multiplication proof.

- **`constructor`**: Builds values of inductive types. For example, `constructor` might construct an `Equiv.Perm X` by providing the forward and inverse functions.

These tactics form the core toolkit for interactive theorem proving in Lean. Each tactic has precise semantics and helps build formal proofs step by step.

12. References

John B. Fraleigh, Victor J. Katz, *A First Course in Abstract Algebra*, Addison–Wesley, 2003, Section 16 (Group Actions).